

JSPM's

Rajarshi Shahu College of Engineering Tathawade Pune

Electronics & Telecommunication Dept

S. Y. B. Tech

Academic Year 2020-21

Subject Name: Digital System Design

Unit-1

Combinational logic Design

Number System

- 1) Decimal number
- 2) Binary
- 3) Octal number
- 4) Hexadecimal number

1) What is binary of decimal 12

	Quotient	Remainder
12/2	6	0 LSB
6/2	3	0
3/2	1	1
1/2	0	1 MSB
12 = 1100		

MSB- Most significant bit

LSB-Least significant bit

2) What is binary of 23

3) What is binary of 35

Number System

Binary to decimal conversion

1) 11111.11

$$1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$$

$$16 + 8 + 4 + 2 + 1 + 0.5 + 0.25$$

(31.75)

2) 110001.101

3) 1110001

4) 1110011

5) 100001

Decimal to octal conversion

247

quotient

Remainder

247/8

30

7 least significant

30/8

3

6

3/8

0

3 Most significant

247 = 367 octal number

Number System

Decimal	Octal	Binary
0	0	000
1	1	001
2	2	010
3	3	011
4	4	100
5	5	101
6	6	110
7	7	111
8	10	1000
9	11	1001
10	12	1010

Decimal	Octal	Binary
11	13	1011
12	14	1100
13	15	1101
14	16	1110
15	17	1111

1) Convert 1001110 to its octal equivalent

001 001 110
116

2) Convert (0.10100110) to its equivalent octal number

0.101 001 100
0.514

It is highly inconvenient to handle long strings of binary numbers. It may cause error also, therefore, the knowledge of octal number system is very important

Number System

Hexadecimal Number System

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000

Decimal	Hexadecimal	Binary
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Hexadecimal to decimal conversion

3A.2F

$$3 \times 16^1 + 10 \times 16^0 + 2 \times 16^{-1} + 15 \times 16^{-2}$$

58.1836

Binary Arithmetic

Binary Addition

Augend	Addend	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Binary subtraction

Minuend	Subtrahend	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Signed binary number system

1) Sign magnitude

➤ $8 = 01000$



➤ $-8 = 11000$

2) One's complement

$$8 = 01000$$

$$-8 = 10111$$

3) Two's complement

$$8 = 01000$$

$$\begin{aligned} -8 &= 10111 \quad \text{one's complement} \\ &\quad 1 \quad \text{add 1} \\ &= 11000 \end{aligned}$$

Two's Complement Arithmetic

. This 2's complement concept eliminates the need of additional circuits for subtraction .

rather the same adder circuit is used two times to perform subtraction

- 1) Perform binary subtraction using two's complement representation of negative numbers

$$\begin{array}{r} \text{i) } 7 \\ -5 \\ \hline 2 \end{array}$$

7 = 0111

$$\begin{array}{r} 5 = 0101 \\ \text{1's comp of } 5 = 1010 \\ \text{2's comp of } 5 = 1011 \end{array}$$

$$\begin{array}{r} \\ 0111 \\ + 1011 \\ \hline 1\ 0010 \end{array}$$

Discard carry Ans
is positive
= 2

➤ ii) 5

$$5 = 0101$$

$$-7 = 0111$$

$$0101$$

➤ -7

One's complement of 7 1000

$$+ 1001$$

➤ -2 (1110)

$$+ \underline{1}$$

$$\underline{1110}$$

➤

Two's complement of 7 1001

Ans = -2 Ans is Two's
complement form

o

➤ If carry =0, ans is negative and it is in its 2's complement form.

Two's Complement Arithmetic

i) $48 + (-23)$

$$\begin{array}{r} 48 = 0110000 \\ -23 \\ \hline 25 \end{array}$$

$$\begin{array}{r} -23 = 0010111 \\ 1101000 \\ \hline 1101001 \end{array}$$

$$\begin{array}{r} 0110000 \\ 1101001 \\ \hline 1\ 0011001 \end{array}$$

Ans is positive

Ans = 25

Discard carry

ii) $-48 + (-23)$

$$\begin{array}{r} -48 \\ -23 \\ \hline -71 \end{array}$$

$$\begin{array}{r} -48 = 0110000 \\ 1001111 \\ \hline 1010000 \end{array}$$

$$\begin{array}{r} \uparrow -23 = 0010111 \\ 1101000 \\ \hline 1\ 1101001 \end{array}$$

$$\begin{array}{r} 11010000 \\ 11101001 \\ \hline 1\ 10111001 \end{array} \quad \begin{array}{r} 10111001 \\ 01000110 \\ \hline 1\ 01000111 \end{array}$$

Discard carry. Ans is positive but should be negative ie -71

$$\begin{array}{r} -48\ 1010000 \\ -23\ 1101001 \\ \hline 1\ 0111001 \end{array}$$

Sign of answer is positive, wrong output

If the two operands are the same sign, the sign bit of the result is to be compared with sign bit of operands.

In case sign bits are same, the result is correct and is in two's complement form.

If the sign bits are not same there is problem of overflow.

Assignment

- 1) Convert (725.25) octal to its decimal binary and hexadecimal
- 2) Convert (11001011.01110) binary into decimal
- 3) Convert decimal number 61.3 to binary.
- 4) Convert octal number 574 to binary & decimal.
- 5) Convert A92H to octal
- 6) Convert following number to hexadecimal
 - i) $(3608)_8$ ii) $(2262)_{10}$ decimal iii) (10011.1101) binary
- 7) Convert (268.75) decimal to binary, octal and hexadecimal
- 8) Convert the following hexadecimal into their equivalent decimal and octal numbers
 - i) $(23867)_{16}$ ii) $(368170.AB)_{16}$

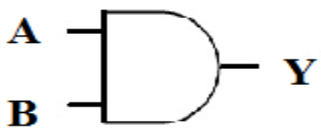
Logic Gates

NOT GATE



$$Y = \overline{A}$$

AND GATE



$$Y = AB$$

OR GATE



$$Y = A + B$$

Truth Table of NOT gate IC 7404

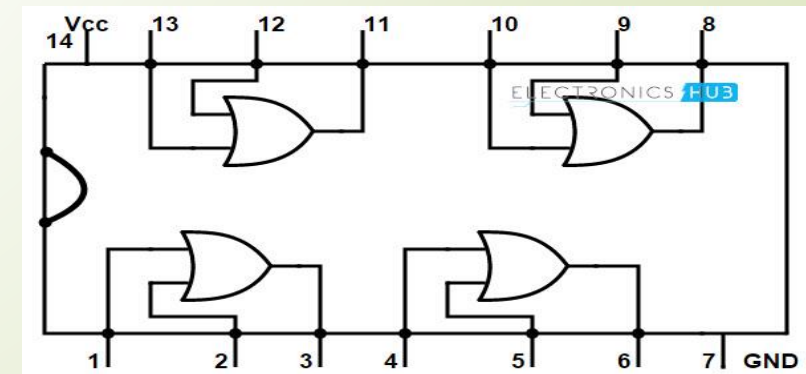
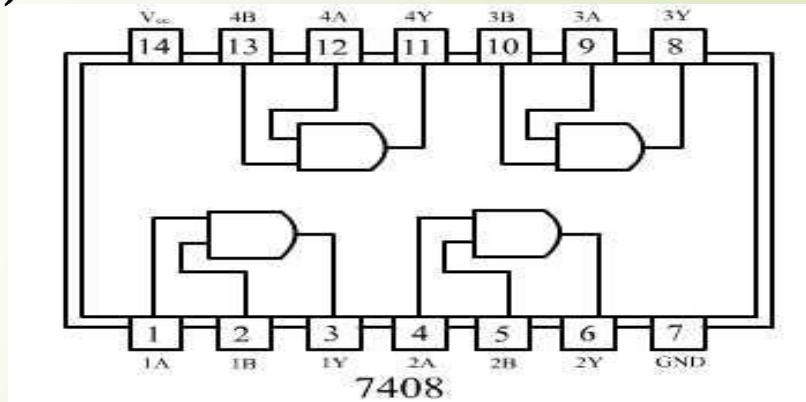
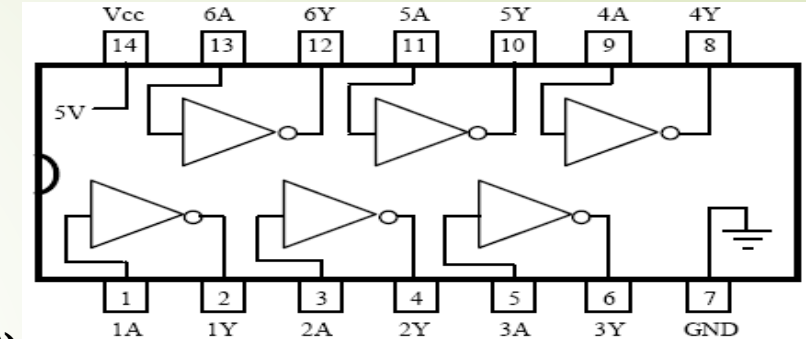
A	Y
0	1
1	0

Truth Table of AND GATE (IC 7408)

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Truth Table of OR GATE (IC 7432)

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



Logic Gates

NAND GATE



$$Y = \overline{AB}$$

NOR GATE



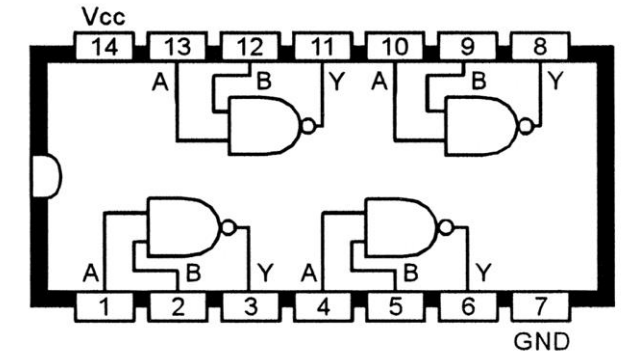
$$Y = \overline{A + B}$$



$$Y = \overline{A}B + A\overline{B}$$

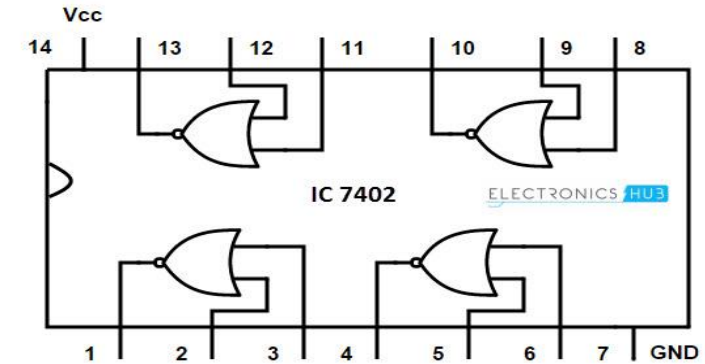
Truth Table of NAND Gate IC 7400

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



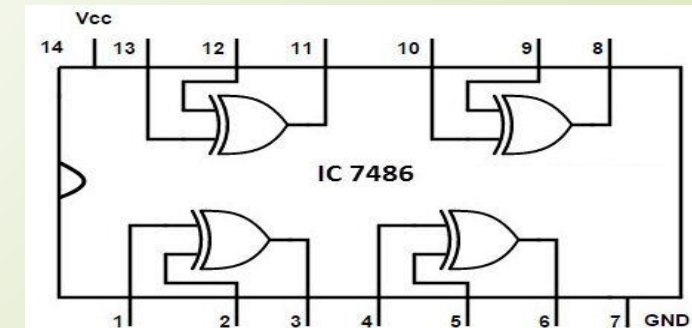
Truth Table of NOR Gate IC 7402

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0



Truth Table of XOR Gate IC 7486

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



Boolean Algebra

In 19th Century an English mathematician George Boole developed rules for manipulations of binary variables, known as boolean algebra

Theorem No.	Theorem
1.1	$A+0=A$
1.2	$A.1=A$
1.3	$A+1=1$
1.4	$A.0=0$
1.5	$A+A=A$
1.6	$A.A=A$
1.8	$A+A'=1$
1.9	$A.(B+C)=AB+AC$
1.10	$A+BC=(A+B)(A+C)$
1.11	$A+AB=A$

- 
- $A+0=A$
 - $A=1, 1+0=1$
 - $A=0, 0+0=0$

Basic Theorem of Boolean Algebra

T1 : Properties of 0

$$(a) 0 + A = A$$

$$(b) 0 A = 0$$

T2 : Properties of 1

$$(a) 1 + A = 1$$

$$(b) 1 A = A$$

Basic Theorem of Boolean Algebra

T3 : Commutative Law

$$(a) A + B = B + A$$

$$(b) A B = B A$$

T4 : Associate Law

$$(a) (A + B) + C = A + (B + C)$$

$$(b) (A B) C = A (B C)$$

T5 : Distributive Law

$$(a) A (B + C) = A B + A C$$

$$(b) A + (B C) = (A + B) (A + C)$$

$$(c) A + A'B = A + B$$

Basic Theorem of Boolean Algebra

T6 : Idempotence (Identity) Law

$$(a) A + A = A$$

$$(b) A A = A$$

T7 : Absorption (Redundance) Law

$$(a) A + A B = A$$

$$(b) A (A + B) = A$$

Basic Theorem of Boolean Algebra

T8 : Complementary Law

(a) $X + X' = 1$

(b) $X \cdot X' = 0$

T9 : Involution

(a) $x'' = x$

T10 : De Morgan's Theorem

(a) $(X + Y)' = X' \cdot Y'$

(b) $(X \cdot Y)' = X' + Y'$

Boolean Algebra

Theorem No.	Theorem
1.12	$A(A+B)=A$
1.13	$A+A'B=(A+B)$
1.14	$A(A'+B)=AB$
1.15	$AB+AB'=A$
1.16	$(A+B)(A+B')=A$
1.17	$AB+A'C=(A+C)(A'+B)$
1.18	$(A+B)(A'+C)=AC+A'B$
1.19	$AB+A'C+BC=AB+A'C$
1.20	$(A+B)(A'+C)(B+C)=(A+B)(A'+C)$
1.21	$(A.B.C)'=A'+B'+C'$
1.22	$(A+B+C)'=A'.B'.C'$

1) Boolean algebraic theorem are used for the Manipulations of logic expression.

Boolean Algebra

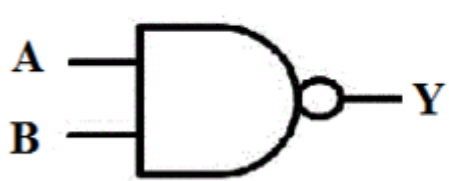
Truth table to prove theorem 1.10

A	B	C	BC	A+BC	A+B	A+C	(A+B).(A+C)
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

Demorgan's theorem

$$\overline{AB} = \overline{A} + \overline{B}$$

NAND GATE

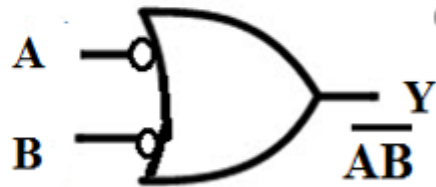


$$Y = \overline{AB}$$

NAND $\equiv$ Bubbled OR



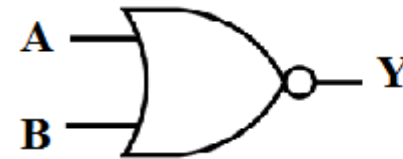
Bubbled OR



$$Y = \overline{A} + \overline{B}$$

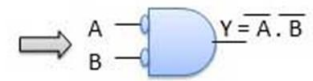
$$\overline{A + B} = \overline{A} \overline{B}$$

NOR GATE

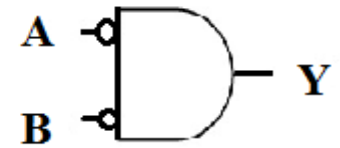


$$Y = \overline{A + B}$$

NOR $\equiv$ Bubbled AND



Bubbled AND



$$Y = \overline{A} \overline{B}$$

A	B	$\overline{AB}$	$\overline{A} + \overline{B}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

A	B	$\overline{A + B}$	$\overline{A} \overline{B}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0


$$\rightarrow Y = A + A'B + AB'$$


$$\Rightarrow Y = A + A'B + AB'$$

$$\Rightarrow = A(1 + B') + A'B$$

$$\Rightarrow = A + A'B$$

$$\Rightarrow = A + B$$

$$1 + B' = 1$$

$$(A + A')(A + B)$$

Boolean Algebra

Conclusion :Reduction of equation before implementation is very important

- Reduction of hardware
- Reduction of size
- Reduction of power requirement
- Reduction of cost

$$\begin{aligned} AC + ABC &= C(A + A'B) \\ &= C(A + B) \end{aligned}$$

Digital Design

Digital circuits are divided into two broad categories

- 1) Combinational logic
- 2) Sequential logic

In combinational circuits, the outputs of any instant of time depend upon the inputs present at that instance of time. This means there is no memory in these circuits.

- 1) Mux
- 2) Demux
- 3) Logic gates

Sequential circuit

- 1) Flip flop
- 2) Shift register
- 3) counter

Hardware Circuit

SOP equation
implemented
using Only NAND
gates

The representation of Eq. is known as sum-of products (SOP) form. This can be realised using AND-OR configuration. This realisation is known as two level realisation. The first level consist of AND gates and the second level consist of OR gate.

By making use of the De-Morgan's Theorem we can write equation as

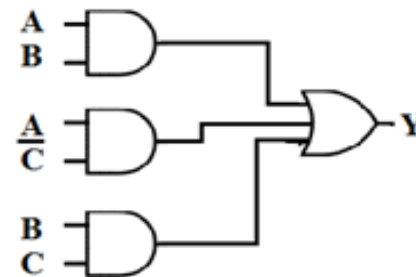
$$Y = AB + A\bar{C} + BC$$

$$\overline{\overline{Y}} = \overline{\overline{AB + A\bar{C} + BC}}$$

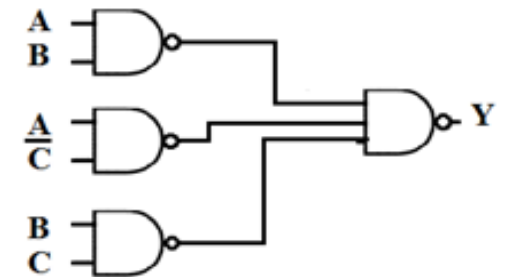
$$\overline{\overline{Y}} = \overline{(\overline{AB}) \cdot (\overline{A\bar{C}}) \cdot (\overline{BC})}$$

a) Hardware circuit using equation

$$Y = AB + A\bar{C} + BC$$



b) Hardware circuit using NAND gate only



Hardware Circuit

Products of Sums form

$$Y = (A + BC) (B + C'A)$$

Theorem $A + BC = (A + B) (A + C)$ using this theorem

$$Y = (A + B) (A + C) (B + C') (B + A)$$

Theorem 1.6 $A.A = A$

$$Y = (A + B) (A + C) (B + C')$$

The representation of above is known as product-of-sums (POS) form. This can be realised using OR-AND gates as shown in Fig.

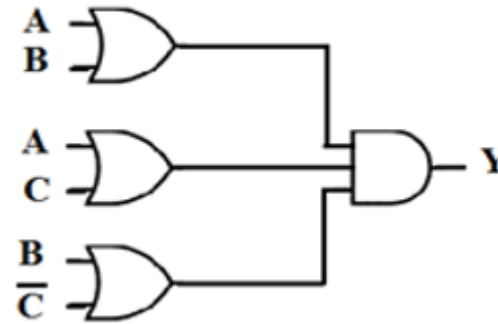
Hardware circuit

$$Y = (A + B) (A + C) (B + \overline{C})$$

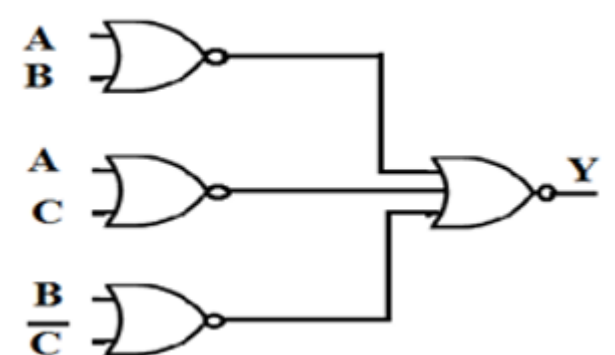
$$\overline{\overline{Y}} = \overline{\overline{(A + B) (A + C) (B + \overline{C})}}$$

$$Y = \overline{\overline{(A + B)} + \overline{\overline{(A + C)}} + \overline{\overline{(B + \overline{C})}}}$$

a)



b)

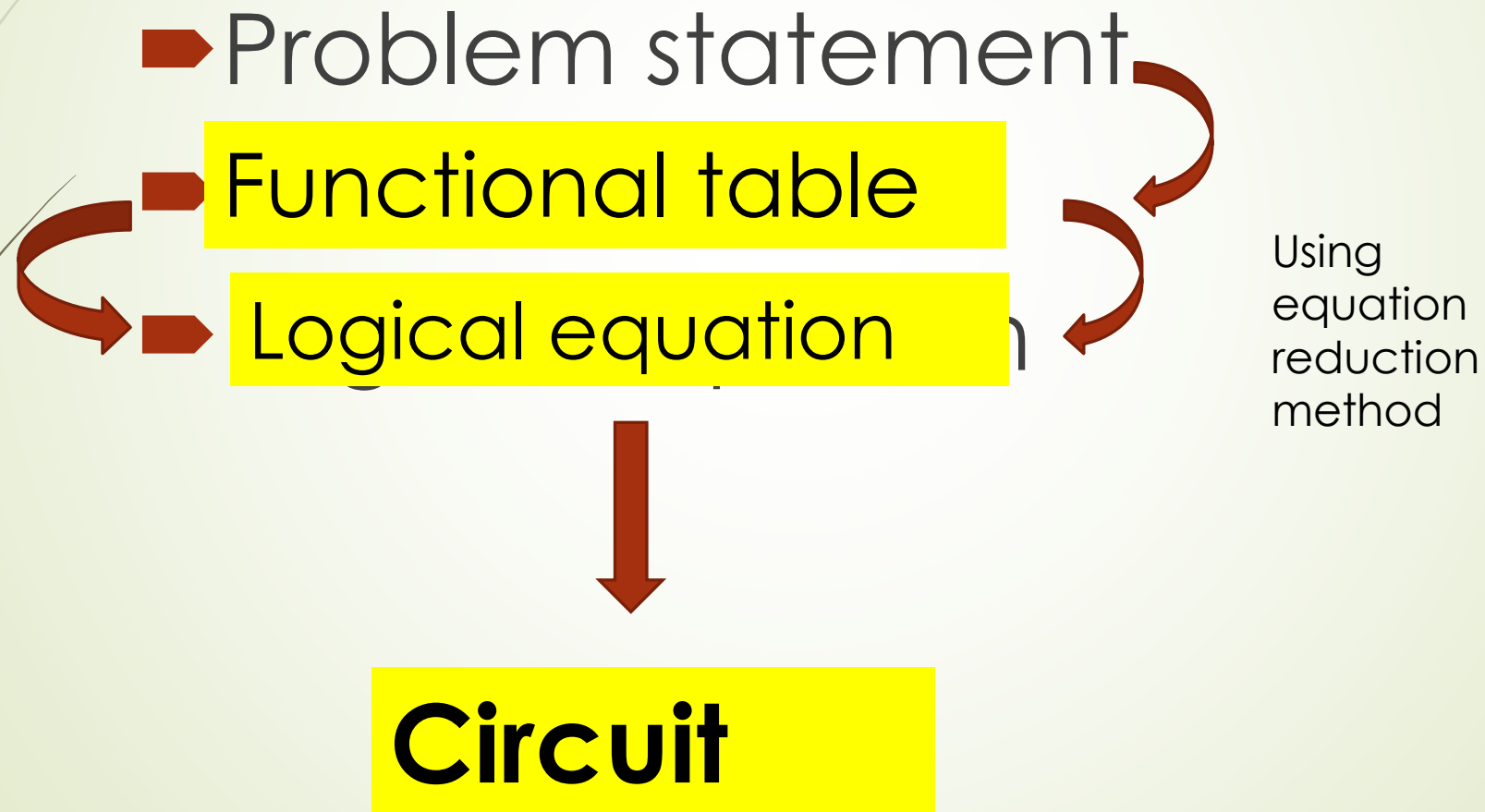


Above equation can be realised using NOR gates only. This realisation is given in Fig. which is two level realisation.

Hence, if we express the equation in POS form we can always design the circuit using only one type of gates (NOR).



To design any digital circuit



Minterm and Maxterm for 4 variable

ABC	Minterm (m)	Maxterm (M)
000	$A'B'C'=m_0$	$A+B+C=M_0$
001	$A'B'C=m_1$	$A+B+C'=M_1$
010	$A'BC'=m_2$	$A+B'+C=M_2$
011	$A'BC=m_3$	$A+B'+C'=M_3$
100	$AB'C'=m_4$	$A'+B+C=M_4$
101	$AB'C=m_5$	$A'+B+C'=M_5$
110	$ABC'=m_6$	$A'+B'+C=M_6$
111	$ABC=m_7$	$A'+B'+C'=M_7$

Minterm

0=complement

1=uncomplement

Maxterm

0=uncomplement

1=complement

Minterm/ Maxterm for 4 variable

ABCD	Minterm	Maxterm
0000	$A'B'C'D'=m_0$	$A+B+C+D=M_0$
0001	$A'B'C'D=m_1$	$A+B+C+D'=M_1$
0010	$A'B'CD'=m_2$	$A+B+C'+D=M_2$
0011	$A'B'CD=m_3$	$A+B+C'+D'=M_3$
0100	$A'BC'D'=m_4$	$A+B'+C+D=M_4$
0101	$A'BC'D=m_5$	$A+B'+C+D'=M_5$
0110	$A'BCD'=m_6$	$A+B'+C'+D=M_6$
0111	$A'BCD=m_7$	$A+B'+C'+D'=M_7$
1000	$AB'C'D'=m_8$	$A'+B+C+D=M_8$
1001	$AB'C'D=m_9$	$A'+B+C+D'=M_9$
1010	$AB'CD'=m_{10}$	$A'+B+C'+D=M_{10}$
1011	$AB'CD=m_{11}$	$A'+B+C'+D'=M_{11}$

ABCD	Minterm	Maxterm
1100	$ABC'D'=m_{12}$	$A'+B'+C+D=M_{12}$
1101	$ABC'D=m_{13}$	$A'+B'+C+D'=M_{13}$
1110	$ABCD'=m_{14}$	$A'+B'+C'+D=M_{14}$
1111	$ABCD=m_{15}$	$A'+B'+C'+D'=M_{15}$

Logical Equation writing from table

Problem Statement :

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

1) SOP (Sum of product)

Write the minterms for which output is 1

$$Y = A'B'C + A'BC' + AB'C' + ABC$$

$$F(A,B,C) = \sum m(1,2,4,7)$$

Canonical SOP (Standard SOP) : SOP equation in which all product terms contains all input variables (complement/uncomplement)

Each individual term in canonical SOP form is called minterm.

Logical Equation writing from table

Problem Statement :

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

2) POS (Product of sum)

$$Y = (A+B+C)(A+B'+C')(A'+B+C')(A'+B'+C)$$

$$F(A,B,C) = \prod M(0,3,5,6)$$

Canonical POS (Standard POS) : POS equation in which all product terms contains all input variables (complement/uncomplement)

Each individual term in canonical POS form is called MAXterm.

SOP and POS are complement of each other.

SOP to Standard SOP

- Following SOP equation convert into canonical SOP equation-
- AND with sum of complement and uncomplete form of missing variable.
- $Y = AB + AC' + BC$
- $Y = AB(C+C') + AC'(B+B') + BC(A+A')$
- $Y = ABC + ABC' + ABC' + AB'C' + ABC + A'BC$
- $Y = ABC + ABC' + AB'C' + A'BC$

POS to Standard POS

- Following equation convert into canonical form
- $Y = (A+B) (A+C) (B+C')$
- ORing with product term of complement and uncomplement form of missing variable
- $= (A+B+C.C') (A+B.B'+C) (A.A'+B+C)$
- $= (A+B+C)(A+B+C')(A+B+C)(A+B'+C)(A+B+C)(A'+B+C)$
- $= (A+B+C)(A+B+C')(A+B'+C)(A'+B+C)$



Kmap

3A.1 Introduction

37

- Simplification of Boolean functions leads to simpler (and usually faster) digital circuits.
- Simplifying Boolean functions using identities is time-consuming and error-prone.
- This special section presents an easy, systematic method for reducing Boolean expressions.

3A.1 Introduction

38

- In 1953, Maurice Karnaugh was a telecommunications engineer at Bell Labs.
- While exploring the new field of digital logic and its application to the design of telephone circuits, he invented a graphical way of visualizing and then simplifying Boolean expressions.
- This graphical representation, now known as a Karnaugh map, or Kmap, is named in his honor.

3A.2 Description of Kmaps and Terminology

- A Kmap is a matrix consisting of rows and columns that represent the output values of a Boolean function.
- The output values placed in each cell are derived from the minterms of a Boolean function.
- A *minterm* is a product term that contains all of the function's variables exactly once, either complemented or not complemented.

3A.2 Description of Kmaps and Terminology

- For example, the minterms for a function having the inputs x and y are: $\bar{x}\bar{y}$, $\bar{x}y$, $x\bar{y}$, and xy
- Consider the Boolean function, $F(x, y) = xy + x\bar{y}$
- Its minterms are:

Minterm	X	Y
$\bar{x}\bar{y}$	0	0
$\bar{x}y$	0	1
$x\bar{y}$	1	0
xy	1	1

3A.2 Description of Kmaps and Terminology

- Similarly, a function having three inputs, has the minterms that are shown in this diagram.

Minterm	X	Y	Z
$\bar{X}\bar{Y}\bar{Z}$	0	0	0
$\bar{X}\bar{Y}Z$	0	0	1
$\bar{X}Y\bar{Z}$	0	1	0
$\bar{X}YZ$	0	1	1
$X\bar{Y}\bar{Z}$	1	0	0
$X\bar{Y}Z$	1	0	1
$XY\bar{Z}$	1	1	0
XYZ	1	1	1

3A.2 Description of Kmaps and Terminology

- A Kmap has a cell for each minterm.
- This means that it has a cell for each line for the truth table of a function.
- The truth table for the function $F(x,y) = xy$ is shown at the right along with its corresponding Kmap.

$$F(X, Y) = XY$$

X	Y	XY
0	0	0
0	1	0
1	0	0
1	1	1

	Y	0	1
X	0	0	0
	1	0	1

3A.2 Description of Kmaps and Terminology

- As another example, we give the truth table and KMap for the function, $F(x,y) = x + y$ at the right.
- This function is equivalent to the OR of all of the minterms that have a value of 1.
- Thus:

$$F(x, y) = x + y = \bar{x}y + x\bar{y} + xy$$

$$F(X, Y) = X + Y$$

X	Y	X+Y
0	0	0
0	1	1
1	0	1
1	1	1

	Y	0	1
X	0	0	1
	1	1	1

3A.3 Kmap Simplification for Two Variables

- ▶ Of course, the minterm function that we derived from our Kmap was not in simplest terms.
 - ▶ That's what we started with in this example.
- ▶ We can, however, reduce our complicated expression to its simplest terms by finding adjacent 1s in the Kmap that can be collected into groups that are powers of two.
- In our example, we have two such groups.
 - Can you find them?

x \ y	0	1
0	0	1
1	1	1

3A.3 Kmap Simplification for Two Variables

- The best way of selecting two groups of 1s from our simple Kmap is shown below.
- We see that both groups are powers of two and that the groups overlap.
- The next slide gives guidance for selecting Kmap groups.

		Y	
		0	1
X	0	0	1
	1	1	1

3A.3 Kmap Simplification for Two Variables

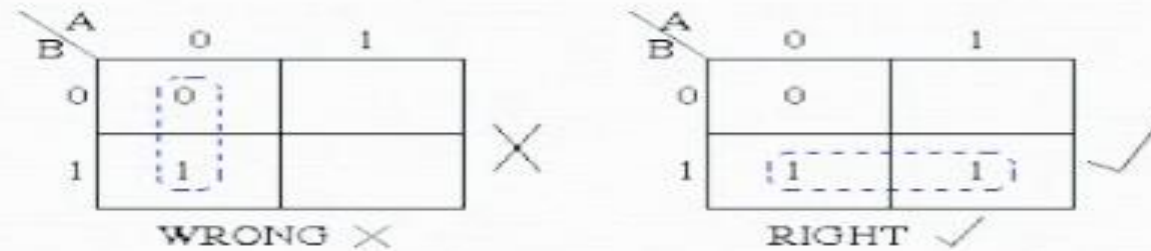
The rules of Kmap simplification are:

- Groupings can contain only 1s; no 0s.
- Groups can be formed only at right angles; diagonal groups are not allowed.
- The number of 1s in a group must be a power of 2 – even if it contains a single 1.
- The groups must be made as large as possible.
- Groups can overlap and wrap around the sides of the Kmap.

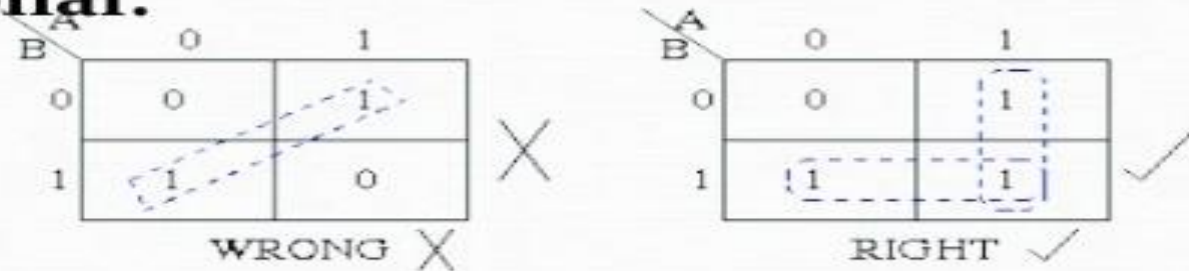
Karnaugh Maps - Rules of Simplification

The Karnaugh map uses the following rules for the simplification of expressions by grouping together adjacent cells containing *ones*.

1.Groups may not include any cell containing a zero.



2.Groups may be horizontal or vertical, but not diagonal.



3. Groups must contain 1, 2, 4, 8, or in general 2^n cells.

That is if $n = 1$, a group will contain two 1's since $2^1 = 2$.

If $n = 2$, a group will contain four 1's since $2^2 = 4$.

A B	0	1
0	1	1
1	0	0

Group of 2

RIGHT ✓

AB C	00	01	11	10
0	0	1	1	1
1	0	0	0	0

Group of 3

WRONG ✗

A B	0	1
0	1	1
1	1	1

Group of 4

RIGHT ✓

AB C	00	01	11	10
0	1	1	1	1
1	0	0	0	1

Group of 5

WRONG ✗

4. Each group should be as large as possible.

$\backslash AB$ C	00	01	11	10
0	1	1	1	1
1	0	0	1	1

RIGHT ✓

$\backslash AB$ C	00	01	11	10
0	1	1	1	1
1	0	0	1	1

WRONG ✗

(Note that no Boolean laws broken,
but not sufficiently minimal)

5. Each cell containing a one must be in at least one group.

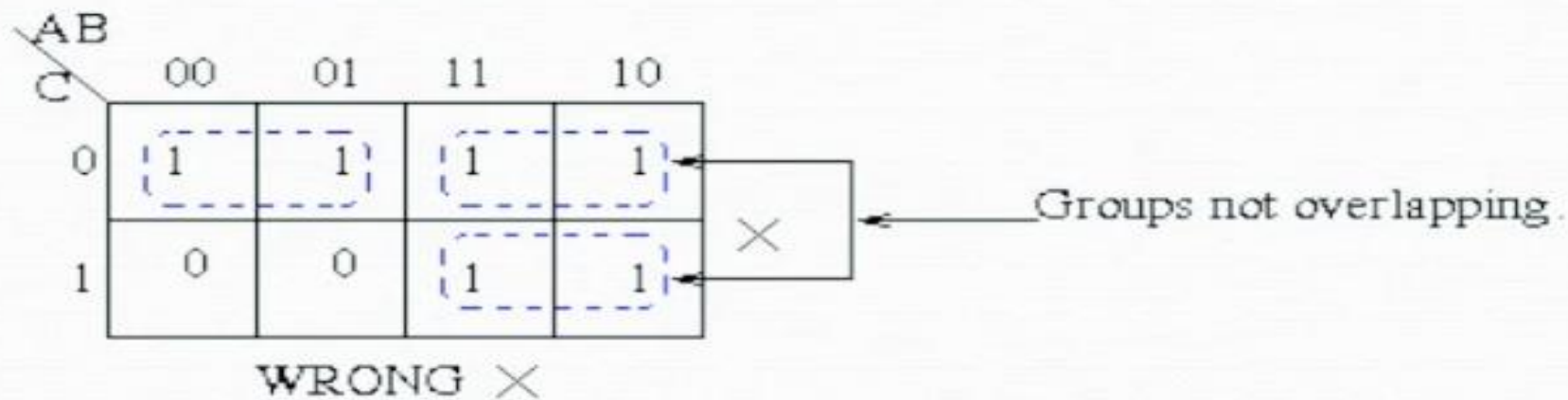
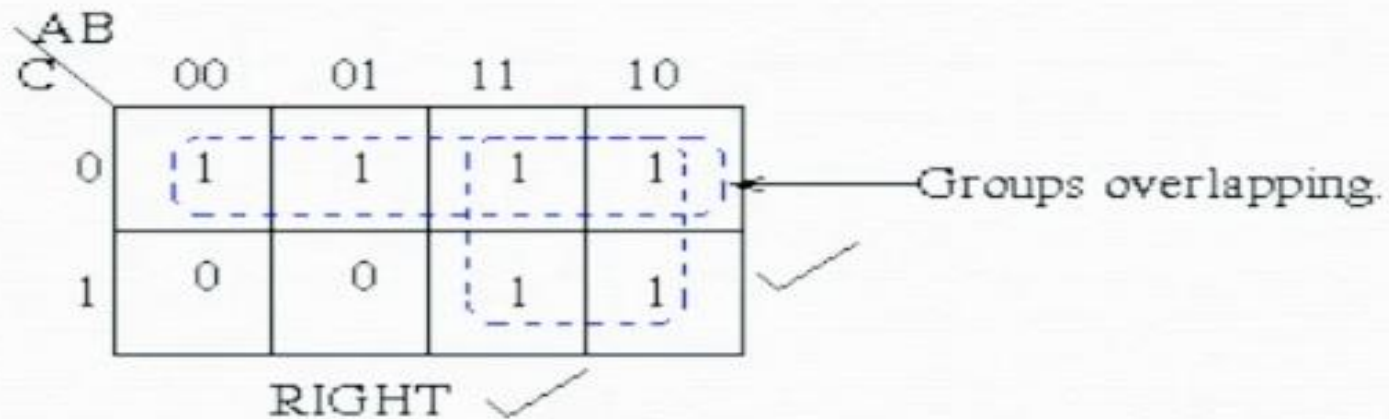
$\backslash AB$ C	00	01	11	10
0	0	0	1	1
1	0	0	0	1

Group I

Group II

1 present in at least one group.

6.Groups may overlap.



7. Groups may wrap around the table.

The leftmost cell in a row may be grouped with the rightmost cell and the top cell in a column may be grouped with the bottom cell.

AB \ C	00	01	11	10
0	1	1	1	1
1	1		1	1

8. There should be as few groups as possible, as long as this does not contradict any of the previous rules.

$\overline{C}/AB$	00	01	11	10
0	1	1	1	1
1	0	0	1	1

RIGHT ✓

$\overline{C}/AB$	00	01	11	10
0	1	1	1	1
1	0	0	1	1

WRONG ✗

Summary:

- No zeros allowed.
- No diagonals.
- Only power of 2 number of cells in each group.
- Groups should be as large as possible.
- Every one must be in at least one group.
- Overlapping allowed.
- Wrap around allowed.
- Fewest number of groups possible.

3A.3 Kmap Simplification for Three Variables

- A Kmap for three variables is constructed as shown in the diagram below.
- We have placed each minterm in the cell that will hold its value.
- Notice that the values for the yz combination at the top of the matrix form a pattern that is not a normal binary sequence.

x \ yz	yz			
	00	01	11	10
0	$\bar{x}\bar{y}\bar{z}$	$\bar{x}\bar{y}z$	$\bar{x}yz$	$\bar{x}y\bar{z}$
1	$x\bar{y}\bar{z}$	$x\bar{y}z$	xyz	$xy\bar{z}$

3A.3 Kmap Simplification for Three Variables

- Thus, the first row of the Kmap contains all minterms where x has a value of zero.
- The first column contains all minterms where y and z both have a value of zero.

x \ yz	yz			
	00	01	11	10
0	$\bar{x}\bar{y}\bar{z}$	$\bar{x}\bar{y}z$	$\bar{x}yz$	$\bar{x}y\bar{z}$
1	$x\bar{y}\bar{z}$	$x\bar{y}z$	xyz	$xy\bar{z}$

3A.3 Kmap Simplification for Three Variables

- Consider the function:

$$F(X, Y) = \bar{X}\bar{Y}Z + \bar{X}YZ + X\bar{Y}Z + XYZ$$

- Its Kmap is given below.
 - What is the largest group of 1s that is a power of 2?

x \ yz	yz			
	00	01	11	10
0	0	1	1	0
1	0	1	1	0

3A.3 Kmap Simplification for Three Variables

- This grouping tells us that changes in the variables x and y have no influence upon the value of the function: They are irrelevant.
- This means that the function,

reduces to $F(x, y, z) = \bar{x}\bar{y}z + \bar{x}yz + x\bar{y}z + xyz$

You could verify this reduction with identities or a truth table.

x \ yz	yz			
	00	01	11	10
0	0	1	1	0
1	0	1	1	0

3A.3 Kmap Simplification for Three Variables

- Now for a more complicated Kmap. Consider the function:

$$F(X, Y, Z) = \bar{X}\bar{Y}\bar{Z} + \bar{X}\bar{Y}Z + \bar{X}YZ + \bar{X}Y\bar{Z} + X\bar{Y}\bar{Z} + XY\bar{Z}$$

- Its Kmap is shown below. There are (only) two groupings of 1s.
 - Can you find them?

x \ yz	yz			
	00	01	11	10
0	1	1	1	1
1	1	0	0	1

3A.3 Kmap Simplification for Three Variables

- In this Kmap, we see an example of a group that wraps around the sides of a Kmap.
- This group tells us that the values of x and y are not relevant to the term of the function that is encompassed by the group.
 - What does this tell us about this term of the function?

What about the green group in the top row?

x \ yz	yz			
	00	01	11	10
0	1	1	1	1
1	1	0	0	1

3A.3 Kmap Simplification for Three Variables

- The green group in the top row tells us that only the value of x is significant in that group.
- We see that it is complemented in that row, so the other term of the reduced function is $\overline{x}$.
- Our reduced function is: $F(x, y, z) = \overline{x} + \overline{z}$

Recall that we had six minterms in our original function!

x \ yz	yz			
	00	01	11	10
0	1	1	1	1
1	1	0	0	1

3A.3 Kmap Simplification for Four Variables

- Our model can be extended to accommodate the 16 minterms that are produced by a four-input function.
- This is the format for a 16-minterm Kmap.

WX \ YZ	YZ			
	00	01	11	10
00	$\bar{W}\bar{X}\bar{Y}\bar{Z}$	$\bar{W}\bar{X}\bar{Y}Z$	$\bar{W}\bar{X}YZ$	$\bar{W}\bar{X}Y\bar{Z}$
01	$\bar{W}X\bar{Y}\bar{Z}$	$\bar{W}X\bar{Y}Z$	$\bar{W}XYZ$	$\bar{W}XY\bar{Z}$
11	$WX\bar{Y}\bar{Z}$	$WX\bar{Y}Z$	$WXYZ$	$WXY\bar{Z}$
10	$W\bar{X}\bar{Y}\bar{Z}$	$W\bar{X}\bar{Y}Z$	$W\bar{X}YZ$	$W\bar{X}Y\bar{Z}$

3A.3 Kmap Simplification for Four Variables

- We have populated the Kmap shown below with the nonzero minterms from the function:

$$F(W, X, Y, Z) = \bar{W}\bar{X}\bar{Y}\bar{Z} + \bar{W}\bar{X}\bar{Y}Z + \bar{W}\bar{X}Y\bar{Z}$$

$$+ \bar{W}XY\bar{Z} + W\bar{X}\bar{Y}\bar{Z} + W\bar{X}\bar{Y}Z + W\bar{X}Y\bar{Z}$$

➤ Can you identify the groups in this Kmap?

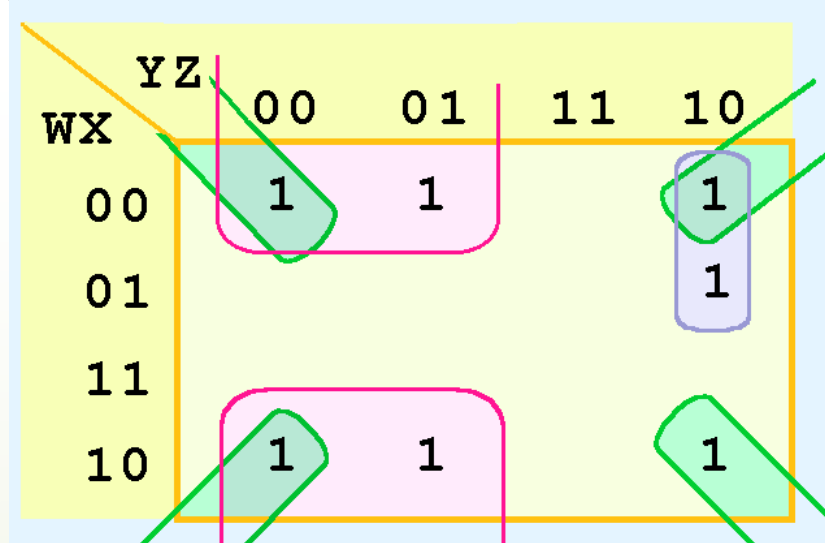
**Recall
that
groups
can
overlap.**

WX \ YZ	YZ			
	00	01	11	10
00	1	1		1
01				1
11				
10	1	1		1

3A.3 Kmap Simplification for Four Variables

- Our three groups consist of:
 - A purple group entirely within the Kmap at the right.
 - A pink group that wraps the top and bottom.
 - A green group that spans the corners.
- Thus we have three terms in our final function:

$$F(W, X, Y, Z) = \bar{W}\bar{Y} + \bar{X}\bar{Z} + \bar{W}Y\bar{Z}$$



3A.3 Kmap Simplification for Four Variables

- It is possible to have a choice as to how to pick groups within a Kmap, while keeping the groups as large as possible.
- The (different) functions that result from the groupings below are logically equivalent.

WX \ YZ	YZ			
	00	01	11	10
00	1		1	
01	1		1	1
11	1			
10	1			

WX \ YZ	YZ			
	00	01	11	10
00	1		1	
01	1		1	1
11	1			
10	1			

3A.6 Don't Care Conditions

66

- Real circuits don't always need to have an output defined for every possible input.
 - For example, some calculator displays consist of 7-segment LEDs. These LEDs can display $2^7 - 1$ patterns, but only ten of them are useful.
- If a circuit is designed so that a particular set of inputs can never happen, we call this set of inputs a *don't care* condition.
- They are very helpful to us in Kmap circuit simplification.

3A.6 Don't Care Conditions

- In a Kmap, a don't care condition is identified by an X in the cell of the minterm(s) for the don't care inputs, as shown below.
- In performing the simplification, we are free to include or ignore the X 's when creating our groups.

WX \ YZ	YZ			
	00	01	11	10
00	X	1	1	X
01		X	1	
11	X		1	
10			1	

3A.6 Don't Care Conditions

68

- In one grouping in the Kmap below, we have the function:

$$F(W, X, Y, Z) = \bar{W}\bar{Y} + YZ$$

WX \ YZ	YZ			
	00	01	11	10
00	X	1	1	X
01		X	1	
11	X		1	
10			1	

3A.6 Don't Care Conditions

- A different grouping gives us the function:

$$F(W, X, Y, Z) = \bar{W}Z + YZ$$

WX \ YZ	YZ			
	00	01	11	10
00	×	1	1	×
01		×	1	
11	×		1	
10			1	

3A.6 Don't Care Conditions

- The truth table of:

$$F(W, X, Y, Z) = \bar{W}\bar{Y} + YZ$$

differs from the truth table of:

$$F(W, X, Y, Z) = \bar{W}Z + YZ$$

- However, the values for which they differ, are the inputs for which we have don't care conditions.

WX \ YZ				
	00	01	11	10
00	×	1	1	×
01		×	1	
11	×		1	
10			1	

WX \ YZ				
	00	01	11	10
00	×	1	1	×
01		×	1	
11	×		1	
10			1	

3A Conclusion

- Kmaps provide an easy graphical method of simplifying Boolean expressions.
- A Kmap is a matrix consisting of the outputs of the minterms of a Boolean function.
- In this section, we have discussed 2- 3- and 4-input Kmaps. This method can be extended to any number of inputs through the use of multiple tables.

3A Conclusion

Recapping the rules of Kmap simplification:

- Groupings can contain only 1s; no 0s.
- Groups can be formed only at right angles; diagonal groups are not allowed.
- The number of 1s in a group must be a power of 2 – even if it contains a single 1.
- The groups must be made as large as possible.
- Groups can overlap and wrap around the sides of the Kmap.
- Use don't care conditions when you can.

Three-Variable K-Maps

$$f = \sum(0,4) = \overline{B} \overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	0
	1	1	0	0	0

$$f = \sum(4,5) = A \overline{B}$$

		BC			
		00	01	11	10
A	0	0	0	0	0
	1	1	1	0	0

$$f = \sum(0,1,4,5) = \overline{B}$$

		BC			
		00	01	11	10
A	0	1	1	0	0
	1	1	1	0	0

$$f = \sum(0,1,2,3) = \overline{A}$$

		BC			
		00	01	11	10
A	0	1	1	1	1
	1	0	0	0	0

$$f = \sum(0,4) = \overline{A} C$$

		BC			
		00	01	11	10
A	0	0	1	1	0
	1	0	0	0	0

$$f = \sum(4,6) = A \overline{C}$$

		BC			
		00	01	11	10
A	0	0	0	0	0
	1	1	0	0	1

$$f = \sum(0,2) = \overline{A} \overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	1
	1	0	0	0	0

$$f = \sum(0,2,4,6) = \overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	1
	1	1	0	0	1

Three-Variable K-Map Examples

A \ BC	00	01	11	10
	0	1		
1	1		1	1

A \ BC	00	01	11	10
	0	1	1	1
1	1			1

A \ BC	00	01	11	10
	0		1	1
1	1	1		

A \ BC	00	01	11	10
	0		1	
1	1		1	1

A \ BC	00	01	11	10
	0	1	1	1
1		1	1	

A \ BC	00	01	11	10
	0			
1				

Four-Variable K-Maps

AB \ CD	00	01	11	10
	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	0	0
10	1	0	0	0

$$f = \sum(0,8) = \bar{B} \cdot \bar{C} \cdot \bar{D}$$

AB \ CD	00	01	11	10
	00	01	11	10
00	0	0	0	0
01	0	1	0	0
11	0	1	0	0
10	0	0	0	0

$$f = \sum(5,13) = B \cdot \bar{C} \cdot D$$

AB \ CD	00	01	11	10
	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	1	1	0
10	0	0	0	0

$$f = \sum(13,15) = A \cdot B \cdot D$$

AB \ CD	00	01	11	10
	00	01	11	10
00	0	0	0	0
01	1	0	0	1
11	0	0	0	0
10	0	0	0	0

$$f = \sum(4,6) = \bar{A} \cdot B \cdot \bar{D}$$

AB \ CD	00	01	11	10
	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	0	0

$$f = \sum(2,3,6,7) = \bar{A} \cdot C$$

AB \ CD	00	01	11	10
	00	01	11	10
00	0	0	0	0
01	1	0	0	1
11	1	0	0	1
10	0	0	0	0

$$f = \sum(4,6,12,14) = B \cdot \bar{D}$$

AB \ CD	00	01	11	10
	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	0	0	0	0
10	0	0	1	1

$$f = \sum(2,3,10,11) = \bar{B} \cdot C$$

AB \ CD	00	01	11	10
	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

$$f = \sum(0,2,8,10) = \bar{B} \cdot \bar{D}$$

Four-Variable K-Maps

AB \ CD	CD			
	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	0	0	0	0

$$f = \sum(4,5,6,7) = \bar{A} \bullet B$$

AB \ CD	CD			
	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	0
10	0	0	1	0

$$f = \sum(3,7,11,15) = C \bullet D$$

AB \ CD	CD			
	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

$$f = \sum(0,3,5,6,9,10,12,15)$$

$$f = A \otimes B \otimes C \otimes D$$

AB \ CD	CD			
	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

$$f = \sum(1,2,4,7,8,11,13,14)$$

$$f = A \oplus B \oplus C \oplus D$$

AB \ CD	CD			
	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

$$f = \sum(1,3,5,7,9,11,13,15)$$

$$f = D$$

AB \ CD	CD			
	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

$$f = \sum(0,2,4,6,8,10,12,14)$$

$$f = \bar{D}$$

AB \ CD	CD			
	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	1	1
10	0	0	0	0

$$f = \sum(4,5,6,7,12,13,14,15)$$

$$f = B$$

AB \ CD	CD			
	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	0	0
10	1	1	1	1

$$f = \sum(0,1,2,3,8,9,10,11)$$

$$f = \bar{B}$$

Four-Variable K-Maps Examples

AB \ CD	CD			
	00	01	11	10
00	1	1		1
01	1	1		1
11	1	1		1
10	1	1		

AB \ CD	CD			
	00	01	11	10
00	1	1		1
01				1
11				
10	1	1		1

AB \ CD	CD			
	00	01	11	10
00				
01	1	1	1	
11	1	1		1
10	1			

AB \ CD	CD			
	00	01	11	10
00		1	1	
01	1	1	1	1
11	1		1	1
10			1	

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

Four-Variable K-Maps Examples



AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

Four-Variable K-Maps Examples



AB \ CD				
	00	01	11	10
00				
01				
11				
10				

AB \ CD				
	00	01	11	10
00				
01				
11				
10				

AB \ CD				
	00	01	11	10
00				
01				
11				
10				

AB \ CD				
	00	01	11	10
00				
01				
11				
10				

AB \ CD				
	00	01	11	10
00				
01				
11				
10				

AB \ CD				
	00	01	11	10
00				
01				
11				
10				

Design of combinational digital circuits

Steps to design a combinational digital circuit:

- From the problem statement derive the truth table
- From the truth table derive the unsimplified logic expression
- Simplify the logic expression
- From the simplified expression draw the logic circuit

- Example: Design a 3-input (A,B,C) digital circuit that will give at its output (X) a logic 1 only if the binary number formed at the input has more ones than zeros

	Inputs			Output
	A	B	C	X
0	0	0	0	
1	0	0	1	
2	0	1	0	
3	0	1	1	
4	1	0	0	
5	1	0	1	
6	1	1	0	
7	1	1	1	1

Design of combinational digital circuits

Steps to design a combinational digital circuit:

- From the problem statement derive the truth table
- From the truth table derive the unsimplified logic expression
- Simplify the logic expression
- From the simplified expression draw the logic circuit

- Example: Design a 3-input (A,B,C) digital circuit that will give at its output (X) a logic 1 only if the binary number formed at the input has more ones than zeros

	Inputs			Output
	A	B	C	
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

→ $X = \sum (3, 5, 6, 7)$

↓

A \ BC				
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

↓

$$X = AC + AB + BC$$



Design of combinational digital circuits (Cont.)

Example: Design a 4-input (A,B,C,D) digital circuit that will give at its output (X) a logic 1 only if the binary number formed at the input is between 2 and 9 (including).

	Inputs				Output
	A	B	C	D	X
0	0	0	0	0	
1	0	0	0	1	
2	0	0	1	0	
3	0	0	1	1	
4	0	1	0	0	
5	0	1	0	1	
6	0	1	1	0	
7	0	1	1	1	
8	1	0	0	0	
9	1	0	0	1	
10	1	0	1	0	
11	1	0	1	1	
12	1	1	0	0	
13	1	1	0	1	
14	1	1	1	0	
15	1	1	1	1	

Design of combinational digital circuits (Cont.)

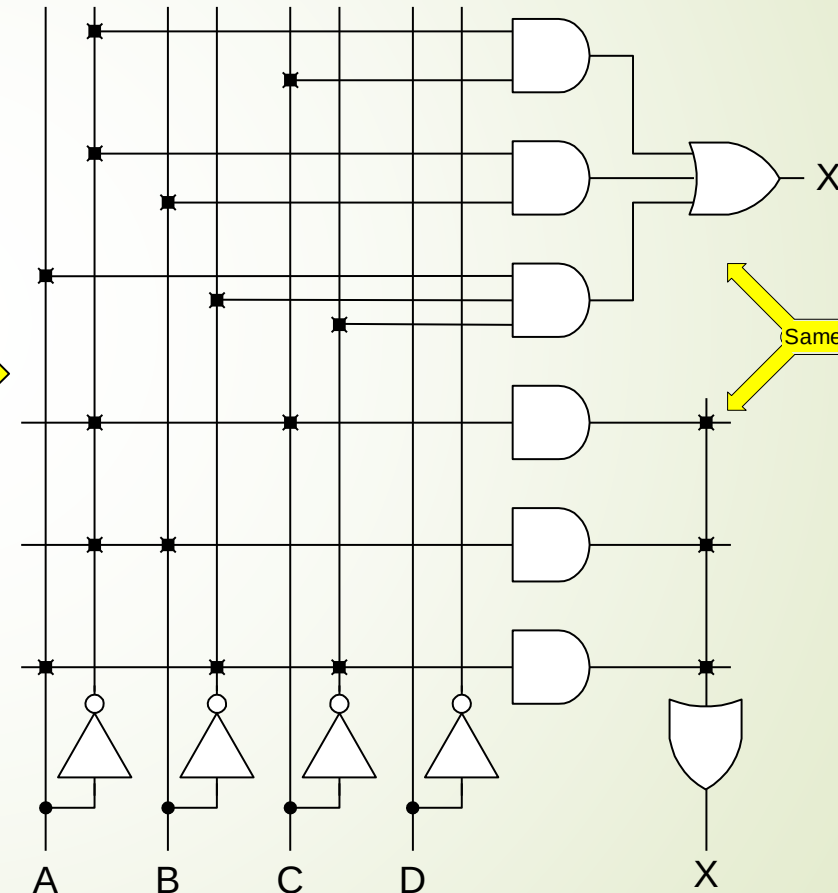
Example: Design a 4-input (A,B,C,D) digital circuit that will give at its output (X) a logic 1 only if the binary number formed at the input is between 2 and 9 (including).

	Inputs				Output
	A	B	C	D	X
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1
10	1	0	1	0	0
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	0

$$X = \sum (2,3,4,5,6,7,8,9)$$

CD \ AB				
	00	01	11	10
00	0	0	1	1
01	1	1	1	1
11	0	0	0	0
10	1	1	0	0

$$X = \bar{A}C + \bar{A}B + A\bar{B}\bar{C}$$



Design of combinational digital circuits (Example)

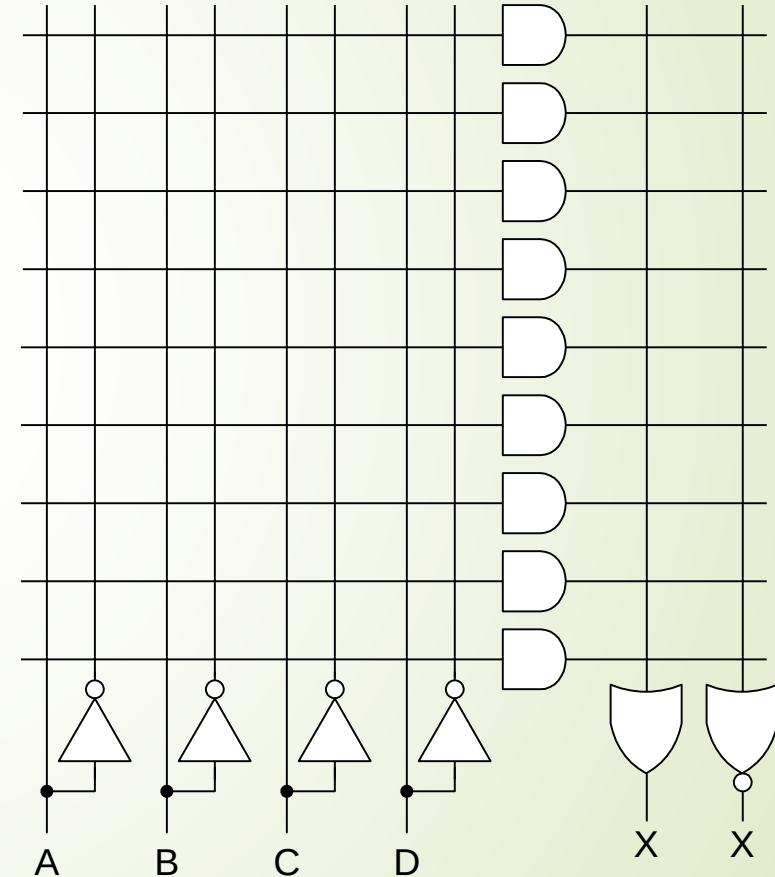
Example: Design a 4-input (A,B,C,D) digital circuit that will give at its output (X) a logic 1 only if there more ones than zeros in the binary number formed at the input.

	Inputs				Output		
	A	B	C	D			
0	0	0	0	0			
1	0	0	0	1			
2	0	0	1	0			
3	0	0	1	1			
4	0	1	0	0			
5	0	1	0	1			
6	0	1	1	0			
7	0	1	1	1			
8	1	0	0	0			
9	1	0	0	1			
10	1	0	1	0			
11	1	0	1	1			
12	1	1	0	0			
13	1	1	0	1			
14	1	1	1	0			
15	1	1	1	1			

X =

AB \ CD				
	00	01	11	10
00				
01				
11				
10				

X =



Reduction of Equation

Boolean algebra X K-map

➤ NAND Gate

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

$$\begin{aligned}Y &= A'B' + A'B + AB' \\&= A'(B' + B) + AB' \\&= A' + AB' \\&= (A' + A)(A' + B') \\&= A' + B' \\Y &= \overline{AB}\end{aligned}$$

	A'	A
B'	1	1
B	1	0

$$\begin{aligned}Y &= A'B' + A'B + A'B' + AB' \\&= A'(B' + B) + B'(A' + A) \\&= A' + B' \\Y &= \overline{AB}\end{aligned}$$

3 Variable equation reduction :

$$Y = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + ABC$$

Using boolean algebra

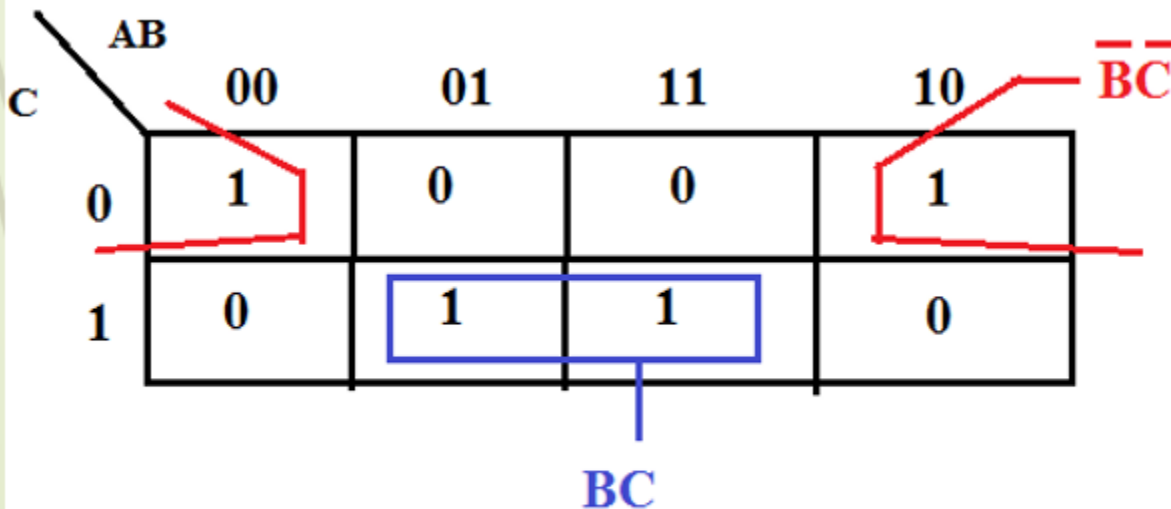
$$Y = (\bar{A} + A)\bar{B}\bar{C} + (\bar{A} + A)BC \quad \text{Theorem 1.7 and 1.2}$$

$$Y = \bar{B}\bar{C} + BC$$

Representation of Truth Table on K-Map

Truth Table

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



$$Y = \bar{B}\bar{C} + BC$$

Equation in terms of canonical SOP

$$Y = A'B'C + A'BC' + AB'C' + ABC$$

Canonocal POS form

$$Y = (A+B+C)(A+B'+C')(A'+B+C')(A'+B'+C)$$

K-map

Two input Variable

B \ A	0	1
0	$A'B'$	AB'
1	$A'B$	AB

B \ A	0	1
0	$A+B$	$A'+B$
1	$A+B'$	$A'+B'$

Three input variable

C \ AB	00	01	11	10
0	$A'B'C'$	$A'BC'$	ABC'	$AB'C'$
1	$A'B'C$	$A'BC$	ABC	$AB'C$

C \ AB	00	01	11	10
0	$A+B+C$	$A+B'+C$	$A'+B'+C$	$A'+B+C$
1	$A+B+C'$	$A+B'+C'$	$A'+B'+C'$	$A'+B+C'$

Four input variable

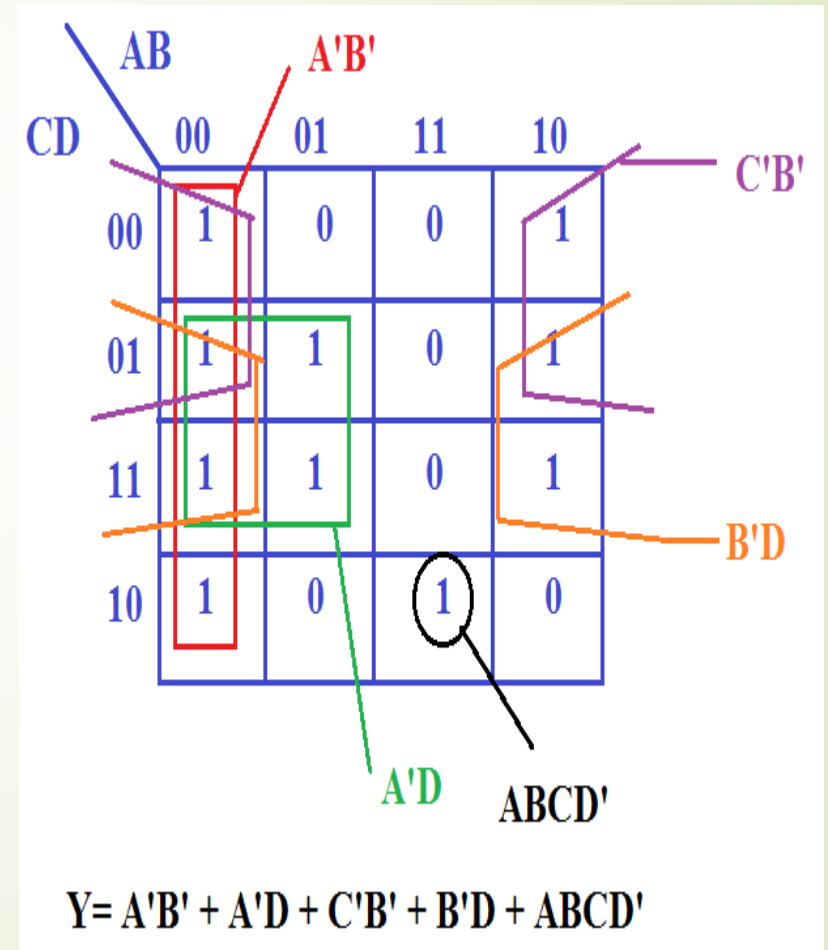
CD \ AB	00	01	11	10
00	$A'B'C'D'$	$A'BC'D'$	$ABC'D'$	$AB'C'D'$
01	$A'B'C'D$	$A'BC'D$	$ABC'D$	$AB'C'D$
11	$A'B'CD$	$A'BCD$	$ABCD$	$AB'CD$
10	$A'B'CD'$	$A'BCD'$	$ABCD'$	$AB'CD'$

CD \ AB	00	01	10	11
00	$A+B+C+D$	$A+B'+C+D$	$A'+B+C+D$	$A'+B'+C+D$
01	$A+B+C+D'$	$A+B'+C+D'$	$A'+B+C+D'$	$A'+B'+C+D'$
11	$A+B+C'D$	$A+B'+C'+D$	$A'+B+C'+D$	$A'+B'+C'+D$
10	$A+B+C'+D$	$A+B'+C'+D$	$A'+B+C'+D$	$A'+B'+C'+D$

K-Map

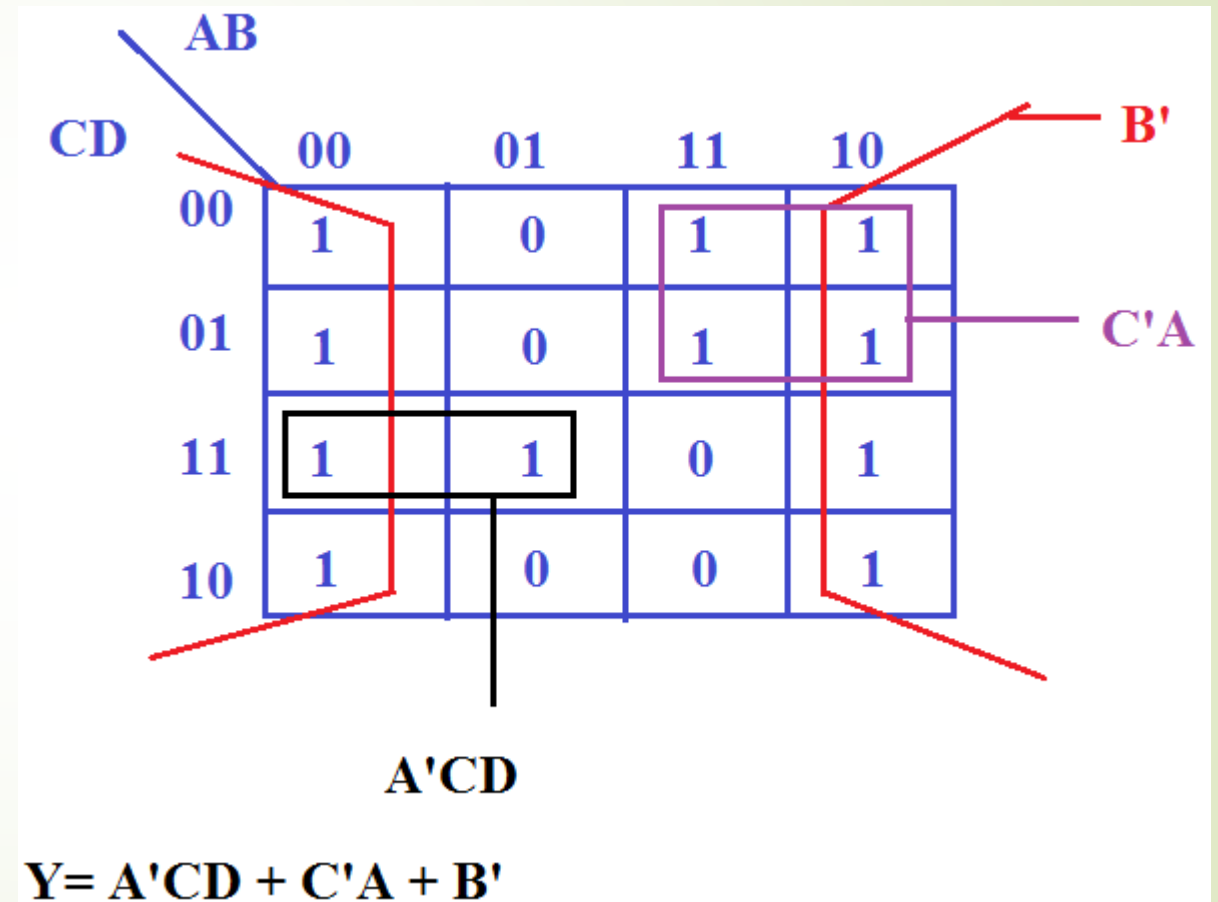
- 1) Minimize the four variable logic function using k-map
 $f(A,B,C,D) = \sum m(0,1,2,3,5,7,8,9,11,14)$

	00	01	11	10
00	0	4	12	8
01	1	5	13	9
11	3	7	15	11
10	2	6	14	10



Getting reduced equation using K-Map

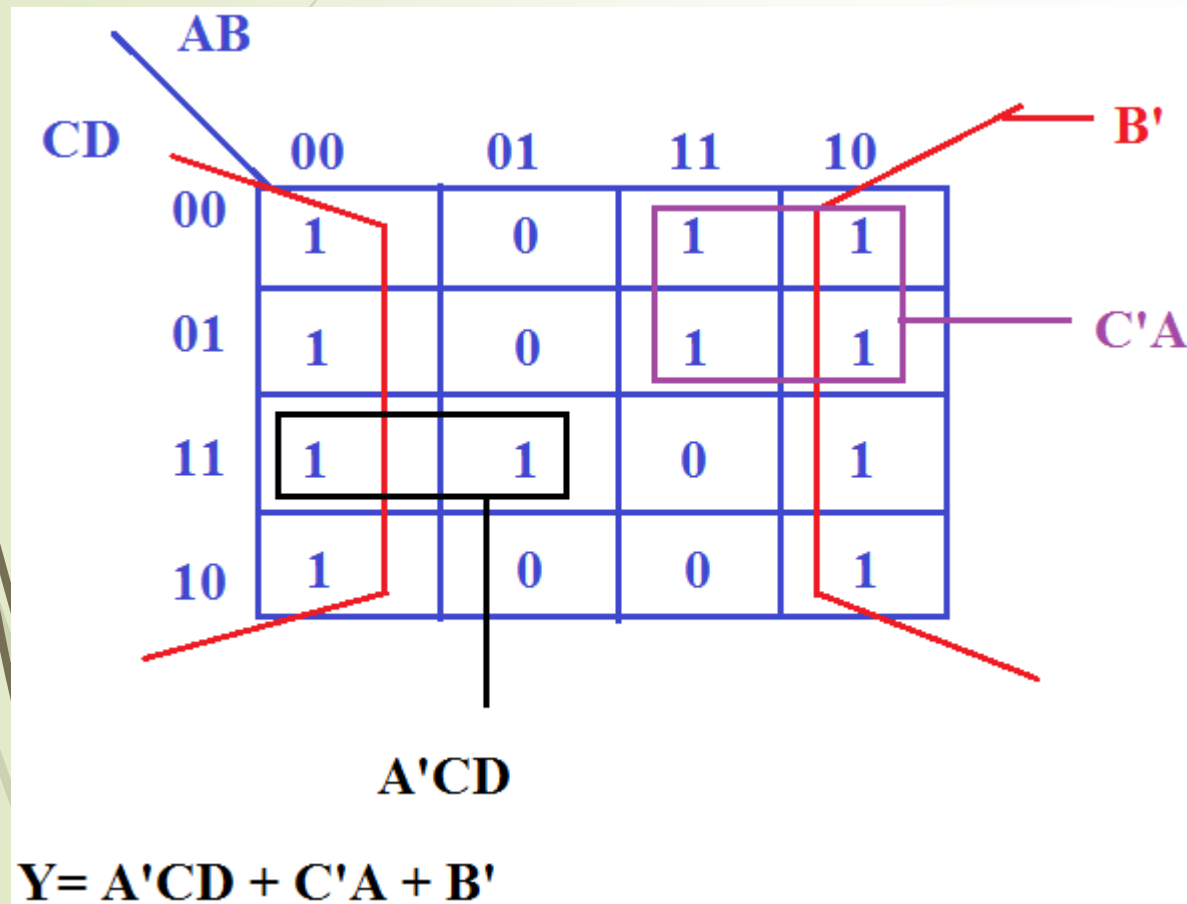
A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	1	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0



K-Map

Minimize the four variable logic function

$$f(A,B,C,D) = ABC'D + A'BCD + A'B'C' + A'B'D' + AC' + AB'C + B'$$



1. Enter 1 in the cell with $A=1, B=1, C=0, D=1$ corresponding to the minterm $ABC'D$.
2. Enter 1 in the cell with $A=0, B=1, C=1, D=1$ corresponding to the minterm $A'BCD$
3. Enter 1's in the two cells with $A=0, B=0, C=0$ corresponding to the term $A'B'C'$.
4. Enter 1's in the two cells with $A=0, B=0, D=0$ (one of there is already entered) corresponding to the term $A'B'D'$.
5. Enter 1's in the two cells with $A=1, B=0, C=1$ corresponding to the term $AB'C$.
6. Enter 1's in the four cells with $A=1, C=0$ (one of them is already entered) corresponding to the term AC' .
7. Enter 1's in the eight cells with $B=0$ (all of them except one have already been entered) corresponding to the term B' .

K-map

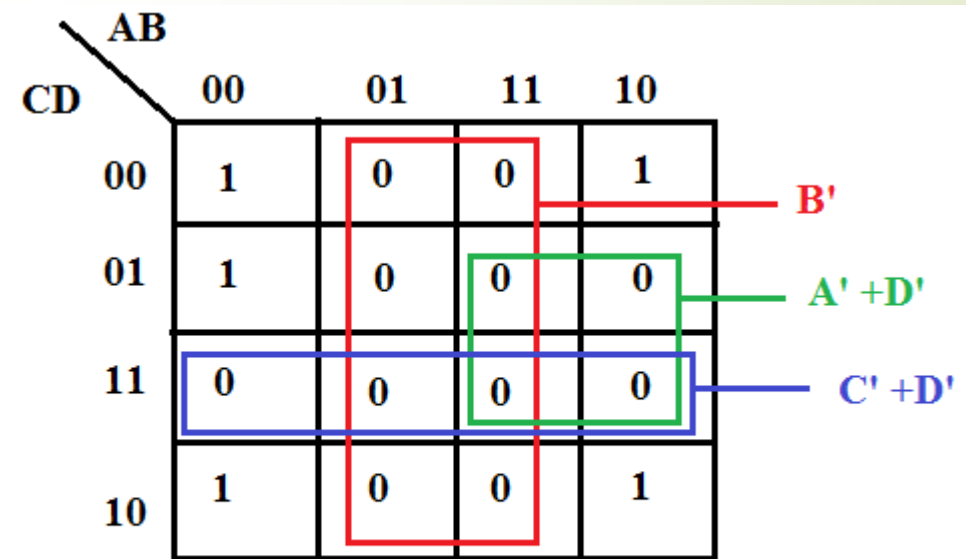
Example Minimise the four variable logic function

$$F(A,B,C,D) = (A+B+C'+D') (A'+C+D') (A'+B+C'+D') (B'+C) (B'+C') (A+B') (B'+D')$$

Convert POS in Standard POS.

Solution : The k-map cells in which 0's are to be entered corresponding to each are given in table . even if a cell is involved in more than one terms, a 0 is to be entered only once.

Term	Cell(s) with 0'
$A + B + C' + D'$	$A=0, B=0, C=1, D=1$
$A' + C + D'$	$A=1, C=0, D=1$
$A' + B + C' + D'$	$A=1, B=0, C=1, D=1$
$B' + C$	$B=1, C=0$
$A + B'$	$A=0, B=1$
$B' + D'$	$B=1, D=1$

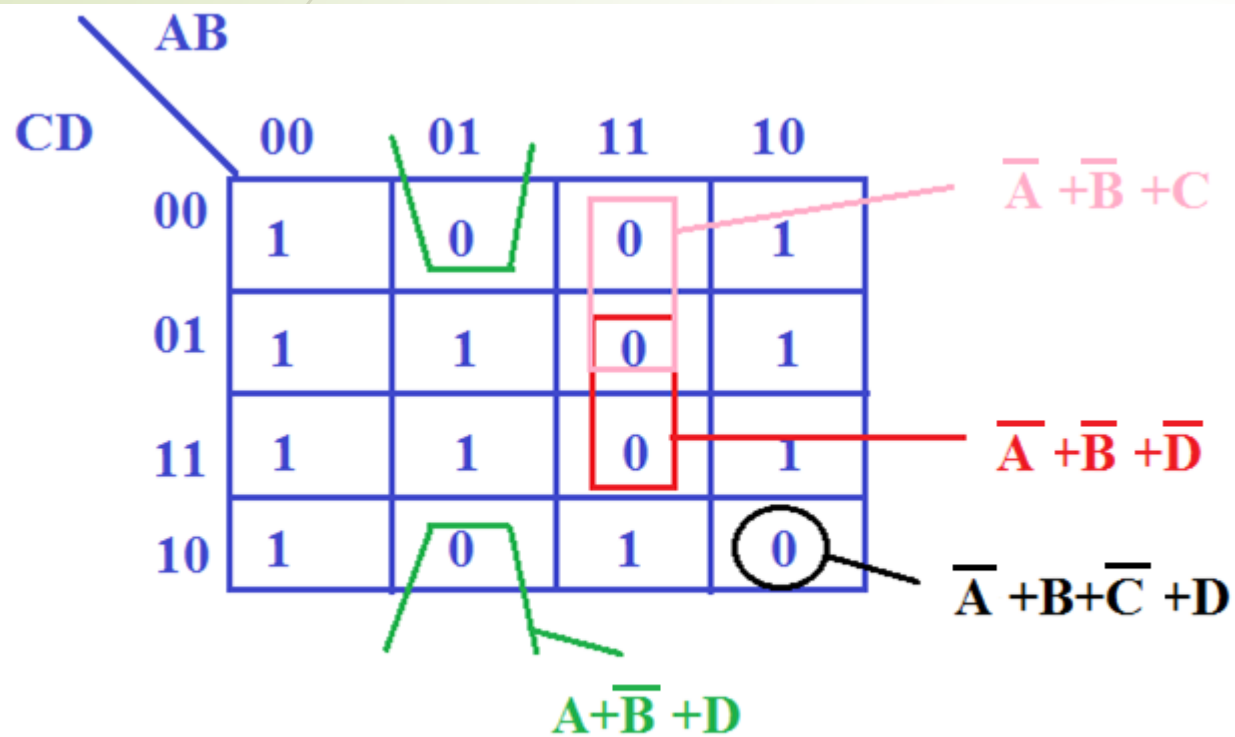


$$Y = B'(A' + D')(C' + D')$$

K-Map

Minimize logic function

$F(A,B,C,D) = \prod M(4,6,10,12,13,15)$ in POS form.



$$Y = (A + \bar{B} + D)(\bar{A} + \bar{B} + C)(\bar{A} + \bar{B} + \bar{D})(\bar{A} + B + \bar{C} + D)$$

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Don't care conditions

$$1) f(A,B,C,D) = \sum m(1,3,7,11,15) + d(0,2,5)$$

AB

CD

00	00	01	11	10
00	X ₀	0 ₄	0 ₁₂	0 ₈
01	1 ₁	X ₅	0 ₁₃	0 ₉
11	1 ₃	1 ₇	1 ₁₅	1 ₁₁
10	X ₂	0 ₆	0 ₁₄	0 ₁₀

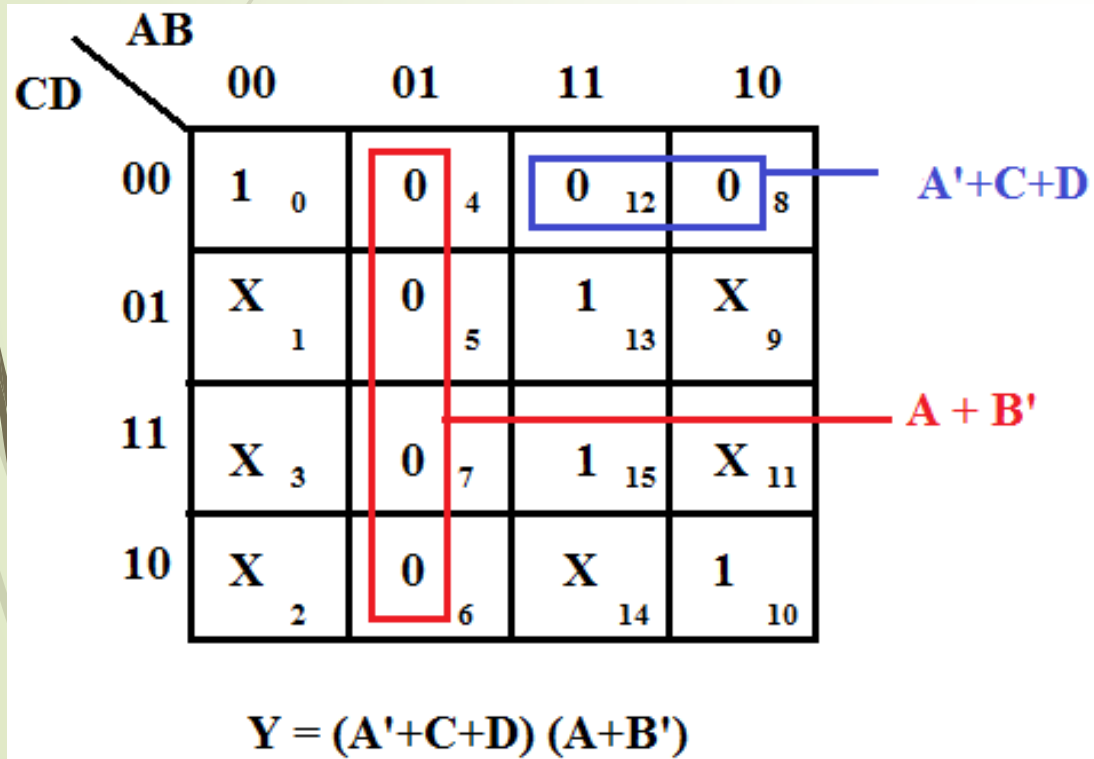
A'D

CD

A	B	C	D	Y
0	0	0	0	X
0	0	0	1	1
0	0	1	0	X
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Don't care conditions

Example 2 In terms of Maxterms and don't care conditions. For example
 $F(A,B,C,D) = \prod M(4,5,6,7,8,12) \cdot d(1,2,3,9,11,14)$



A	B	C	D	Y
0	0	0	0	1
0	0	0	1	X
0	0	1	0	X
0	0	1	1	X
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	X
1	0	1	0	1
1	0	1	1	X
1	1	0	0	0
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1